Cash Receipt Feature - Implementation Task Checklist

Feature: Cash Receipt Management & Tally Integration
Document Version: 1.0
Date: February 20, 2026
Estimated Time: 26 hours (3-4 working days)
Reference: FEATURE_CASH_RECEIPT_IMPLEMENTATION.md

Progress Summary

Overall Progress: 0 / 71 tasks completed (0%)

Phase Progress

- ☐ Setup & Prerequisites: 0 / 3 (0%)
- ☐ Database Layer: 0 / 12 (0%)
- ☐ Backend Layer: 0 / 9 (0%)
- ☐ Frontend - HTML: 0 / 8 (0%)
- ☐ Frontend - CSS: 0 / 6 (0%)
- ☐ Frontend - JavaScript: 0 / 15 (0%)
- ☐ Integration & Testing: 0 / 10 (0%)
- ☐ Deployment: 0 / 8 (0%)

📋 Implementation Checklist

Phase 0: Setup & Prerequisites

Estimated Time: 30 minutes

- ☐ **PRE-1** Review FEATURE_CASH_RECEIPT_IMPLEMENTATION.md thoroughly
- ☐ **PRE-2** Set up local development environment (if not already done)
- ☐ **PRE-3** Create feature branch in git: `git checkout -b feature/cash-receipt`

> **Tip:** Use consistent naming for easy tracking

Phase 1: Database Layer

Estimated Time: 1 hour 45 minutes

Table Creation

- ☐ **DB-1** Open Supabase SQL Editor
- ☐ **DB-2** Create `cash_receipts` table with all fields

> Copy schema from implementation doc lines 185-222

- [ ] **DB-3** Add UNIQUE constraint: `(source_table, payment_reference_id)`
- [ ] **DB-4** Add CHECK constraint: `check_source_consistency`

> Prevents payment_source/source_table mismatch

- [ ] **DB-5** Add CHECK constraint for status: `('pending', 'processed')`
- [ ] **DB-6** Add CHECK constraint for payment_source values
- [ ] **DB-7** Test table creation: `SELECT * FROM cash_receipts LIMIT 1;`

> Should return 0 rows but no error

Indexes

- [ ] **DB-8** Create index: `idx_cash_receipts_user_id`
- [ ] **DB-9** Create index: `idx_cash_receipts_status`
- [ ] **DB-10** Create index: `idx_cash_receipts_payment_date`
- [ ] **DB-11** Create index: `idx_cash_receipts_payment_source`
- [ ] **DB-12** Create index: `idx_cash_receipts_reference`
- [ ] **DB-13** Create UNIQUE index: `idx_unique_tally_receipt_number` (case-insensitive)

> Uses UPPER() to prevent 'CASH-001' vs 'cash-001' duplicates

- [ ] **DB-14** Create composite index: `idx_cash_receipts_user_date`
- [ ] **DB-15** Create performance indexes on source tables:
  - [ ] `idx_approved_bills_cash_payments`
  - [ ] `idx_ugrai_payment_cash`
- [ ] **DB-16** Verify indexes created: `SELECT indexname FROM pg_indexes WHERE tablename = 'cash_receipts';`

RLS Policies

- [ ] **DB-17** Enable RLS: `ALTER TABLE cash_receipts ENABLE ROW LEVEL SECURITY;`
- [ ] **DB-18** Create SELECT policy: "Admins and accountants can view all cash receipts"

> No user_id filtering - shared team data

- [ ] **DB-19** Create INSERT policy: "Admins and accountants can insert cash receipts"
- [ ] **DB-20** Create UPDATE policy: "Admins and accountants can update cash receipts"
- [ ] **DB-21** Test RLS policies: `SELECT policyname FROM pg_policies WHERE tablename = 'cash_receipts';`

> Should show 3 policies

Triggers

- [ ] **DB-21.1** Check if `update_updated_at_column()` function already exists

```
SELECT proname FROM pg_proc WHERE proname = 'update_updated_at_column';
```

> If it exists, skip DB-21.2; if not, create it

- [ ] **DB-21.2** Create trigger function (if not exists): `update_updated_at_column()`

> Copy from implementation doc - automatically updates updated_at timestamp

- [ ] **DB-21.3** Create trigger: `update_cash_receipts_updated_at`

> Calls trigger function before each update

- [ ] **DB-21.4** Test trigger: Update a test record and verify updated_at changes

Database Functions

- [ ] **DB-22** Create function: `get_all_cash_payments(p_start_date DATE, p_end_date DATE)`

> Copy from implementation doc lines 300-427
> Note: NO p_user_id parameter (shared data architecture)

- [ ] **DB-23** Verify function signature in RETURNS TABLE matches all 12 columns
- [ ] **DB-24** Grant execute permission: `GRANT EXECUTE ON FUNCTION get_all_cash_payments(DATE, DATE) TO authenticated;`
- [ ] **DB-25** Test function execution: `SELECT * FROM get_all_cash_payments('2025-01-01', '2025-12-31') LIMIT 5;`

> Should return cash payments from approved_bills and ugrai_payment_history

Database Testing

- [ ] **DB-TEST-1** Insert test cash receipt (as admin/accountant)

```
INSERT INTO cash_receipts (user_id, payment_source, source_table,
payment_reference_id,
  bill_number, bill_date, retailer_name, payment_amount, payment_date,
  tally_receipt_number, status)
VALUES (auth.uid(), 'supply', 'approved_bills', gen_random_uuid(),
  'TEST-001', '2025-10-15', 'Test Store', 1000.00, '2025-10-16',
  'CASH-TEST-001', 'processed');
```

- [ ] **DB-TEST-2** Verify constraint prevents mismatched source: Try inserting `payment_source='supply'` with `source_table='ugrai_payment_history'` (should fail)
- [ ] **DB-TEST-3** Verify case-insensitive unique constraint: Try inserting 'CASH-TEST-001' and 'cash-test-001' (second should fail)
- [ ] **DB-TEST-4** Delete test data: `DELETE FROM cash_receipts WHERE bill_number = 'TEST-001';`

Phase 2: Backend Layer

Estimated Time: 2 hours 35 minutes

File: `supabase-service.js`

Utility Functions

- [ ] **BE-1** Add `escapeHtml(unsafe)` utility function

  > Prevents XSS attacks - implementation doc lines 687-703

- [ ] **BE-2** Test escapeHtml: Console test with special characters

Core Methods

- [ ] **BE-3** Add `getAllCashPayments(startDate = null, endDate = null)` method

  > Implementation doc lines 708-727

  - [ ] Format dates to YYYY-MM-DD
  - [ ] Call RPC: `get_all_cash_payments`
  - [ ] Pass parameters: `{ p_start_date, p_end_date }`
  - [ ] NO p_user_id parameter (shared data)
  - [ ] Handle null dates (fetch all)
  - [ ] Error handling with try-catch
  - [ ] Console logging for debugging

- [ ] **BE-4** Add `submitCashReceipt(receiptData)` method

  > Implementation doc lines 732-887

  - [ ] Validate required parameters (paymentId, source, sourceTable, etc.)
  - [ ] Map sourceTable validation (security check)
  - [ ] Check for existing receipt using `.maybeSingle()`
  - [ ] If existing: UPDATE with new receipt number
  - [ ] If new: INSERT new cash receipt
  - [ ] Set `user_id: this.userId` (audit trail, NOT for filtering)
  - [ ] Handle duplicate receipt number error (code 23505)
  - [ ] Return updated/created record
  - [ ] Error handling with try-catch

- [ ] **BE-5** Add `hasAdminOrAccountantRole()` method

  > Implementation doc lines 893-915

  - [ ] Query user_roles table
  - [ ] Check for 'admin' or 'accountant' role
  - [ ] Return boolean

- [ ] Error handling returns false

Backend Testing

- [ ] **BE-TEST-1** Test `hasAdminOrAccountantRole()` in browser console

```
const hasRole = await
appState.supabaseService.hasAdminOrAccountantRole();
console.log('Has admin/accountant role:', hasRole); // Should be true for
admin
```

- [ ] **BE-TEST-2** Test `getAllCashPayments()` without date filters

```
const payments = await appState.supabaseService.getAllCashPayments();
console.log('All payments:', payments); // Should return array
```

- [ ] **BE-TEST-3** Test `getAllCashPayments()` with date range

```
const payments = await appState.supabaseService.getAllCashPayments('2025-
01-01', '2025-01-31');
console.log('January payments:', payments);
```

- [ ] **BE-TEST-4** Test `escapeHtml()` with malicious input

```
const escaped = appState.supabaseService.escapeHtml('<script>alert("XSS")
</script>');
console.log('Escaped:', escaped); // Should show &lt;script&gt;...
```

Phase 3: Frontend - HTML Structure

Estimated Time: 1 hour 45 minutes

File: `supply.html`

Tab Navigation

- [ ] **FE-HTML-1** Add Cash Receipt tab button to main navigation

> Add after Analysis tab in `#main-processes` div

```

```

```
<button class="heading-tab" data-process="cash-receipt" id="cash-receipt-tab"
  style="display: none;">CASH RECEIPT</button>
```

> Hidden by default, shown via JavaScript

Dashboard Container

- [ ] **FE-HTML-2** Create `cash-receipt-dashboard-container` div (after other dashboard containers)

> Set `style="display: none;"` initially

- [ ] **FE-HTML-3** Add heading: `<h2>Cash Receipt Management</h2>`

Date Filter Section

- [ ] **FE-HTML-4** Create date filter controls container
  - [ ] Add "Start Date" input: `<input type="date" id="cash-receipt-start-date">`
  - [ ] Add "End Date" input: `<input type="date" id="cash-receipt-end-date">`
  - [ ] Add "Apply Filter" button: `<button id="apply-cash-date-filter-btn">`
  - [ ] Add "Clear" button: `<button id="clear-cash-date-filter-btn">`

Statistics Section

- [ ] **FE-HTML-5** Create statistics display container
  - [ ] Add "Total Payments" display: `<span id="total-cash-payments">0</span>`
  - [ ] Add "Processed" display: `<span id="processed-cash-payments">0</span>`
  - [ ] Add "Pending" display: `<span id="pending-cash-payments">0</span>`
  - [ ] Add "Total Amount" display: `<span id="total-cash-amount">₹0</span>`

Status Filter Section

- [ ] **FE-HTML-6** Create status filter buttons
  - [ ] Add "All" button: `<button class="status-filter-btn active" data-filter="all">`
  - [ ] Add "Pending" button: `<button class="status-filter-btn" data-filter="pending">`
  - [ ] Add "Processed" button: `<button class="status-filter-btn" data-filter="processed">`

Table Section

- [ ] **FE-HTML-7** Create cash receipts table structure

```

```

```
<table>
  <thead>
    <tr>
      <th>Bill Number</th>
      <th>Bill Date</th>
      <th>Retailer Name</th>
      <th>Sector</th>
      <th>Amount</th>
      <th>Payment Date</th>
      <th>Source</th>
      <th>Tally Receipt #</th>
      <th>Status</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody id="cash-receipts-table-body"></tbody>
</table>
```

- ☐ **FE-HTML-8** Add pagination container: `<div id="pagination-container"></div>`
- ☐ **FE-HTML-9** Add export button: `<button id="export-cash-receipts-btn">📊 Export to Excel</button>`

HTML Testing

- ☐ **FE-HTML-TEST-1** Open supply.html in browser and verify all elements exist (using browser DevTools)
- ☐ **FE-HTML-TEST-2** Check that cash-receipt-dashboard-container is hidden by default
- ☐ **FE-HTML-TEST-3** Verify all input IDs are correct (no typos)

Phase 4: Frontend - CSS Styling

Estimated Time: 1 hour 45 minutes

File: `styles.css` or inline in `supply.html`

Status Badges

- ☐ **FE-CSS-1** Style pending status badge

```
.status-pending {
  background-color: #FEF3C7;
  color: #92400E;
  padding: 4px 8px;
  border-radius: 9999px;
  font-size: 0.75rem;
}
```

- [ ] **FE-CSS-2** Style processed status badge

```
.status-processed {
  background-color: #D1FAE5;
  color: #065F46;
  padding: 4px 8px;
  border-radius: 9999px;
  font-size: 0.75rem;
}
```

Source Badges

- [ ] **FE-CSS-3** Style Supply source badge (blue)

```
.source-supply {
  background-color: #DBEAFE;
  color: #1E40AF;
}
```

- [ ] **FE-CSS-4** Style UGRAI source badge (purple)

```
.source-ugrai {
  background-color: #E9D5FF;
  color: #6B21A8;
}
```

- [ ] **FE-CSS-5** Style UGRAI Partial source badge (orange)

```
.source-ugrai-partial {
  background-color: #FED7AA;
  color: #9A3412;
}
```

Row Highlighting

- [ ] **FE-CSS-6** Style processed rows (light green background)

```
tr.status-processed {
  background-color: #F0FDF4 !important;
}
```

- [ ] **FE-CSS-7** Style pending rows hover effect

```
tr.status-pending:hover {
  background-color: #FEF9E7;
}
```

Filter Buttons

- [ ] **FE-CSS-8** Style active filter button

```
.status-filter-btn.active {
  background-color: #3B82F6;
  color: white;
  border: 2px solid #2563EB;
}
```

Payment Grouping

- [ ] **FE-CSS-9** Style grouped payments (blue left border)

```
tr.grouped-payment {
  border-left: 4px solid #3B82F6;
}
```

Loading/Error States

- [ ] **FE-CSS-10** Style loading spinner
- [ ] **FE-CSS-11** Style error message display

Mobile Responsiveness

- [ ] **FE-CSS-12** Add responsive table styles for mobile (stack columns or horizontal scroll)
- [ ] **FE-CSS-13** Ensure buttons stack properly on mobile
- [ ] **FE-CSS-14** Test on mobile viewport (DevTools device mode)

Phase 5: Frontend - JavaScript Implementation

Estimated Time: 5 hours 55 minutes

File: `scripts.js`

Global State Extension

- [ ] **FE-JS-1** Add cashReceiptData to appState

```
cashReceiptData: {
  allPayments: [],
  filteredPayments: [],
  currentFilter: 'all',
  isLoading: false,
  pagination: { currentPage: 1, itemsPerPage: 50, totalPages: 1 }
}
```

DOM Element References

- [ ] **FE-JS-2** Add DOM element references (in initialization section)
  - [ ] cashReceiptDashboardContainer
  - [ ] cashReceiptStartDate
  - [ ] cashReceiptEndDate
  - [ ] applyDateFilterBtn
  - [ ] clearDateFilterBtn
  - [ ] cashReceiptsTable
  - [ ] totalCashPayments
  - [ ] processedCashPayments
  - [ ] pendingCashPayments
  - [ ] totalCashAmount
  - [ ] exportCashReceiptsBtn
  - [ ] cashReceiptTab

Initialization Function

- [ ] **FE-JS-3** Create `initializeCashReceiptDashboard()` function
  - [ ] Check if user has admin/accountant role
  - [ ] If no role: Hide tab and return
  - [ ] If has role: Show tab
  - [ ] Attach event listeners for date filters
  - [ ] Attach event listeners for status filters
  - [ ] Attach event listener for export button
  - [ ] Load initial payment data
  - [ ] Error handling with try-catch
  - [ ] Console logging for debugging

Data Loading Function

- [ ] **FE-JS-4** Create `loadCashPayments()` function
  - [ ] Check isLoading flag (prevent concurrent loads)
  - [ ] Set isLoading = true
  - [ ] Get date range from inputs

    - [ ] Call `getAllCashPayments(startDate, endDate)`
    - [ ] Store in appState.cashReceiptData.allPayments
    - [ ] Apply current filter
    - [ ] Reset pagination to page 1
    - [ ] Render table
    - [ ] Update statistics
    - [ ] Set isLoading = false
    - [ ] Error handling: Show error in table

Filter Functions

- [ ] **FE-JS-5** Create `applyStatusFilter(filter)` function
    - [ ] Set currentFilter in appState
    - [ ] Filter allPayments based on status
    - [ ] Store in filteredPayments
    - [ ] Update filter button active states
    - [ ] Reset pagination to page 1
    - [ ] Render table

Rendering Function

- [ ] **FE-JS-6** Create `renderCashReceiptsTable()` function
    - [ ] Get current page data from filteredPayments
    - [ ] Clear table body
    - [ ] Group payments by bill_number (for visual grouping)
    - [ ] For each payment:
        - [ ] Create table row
        - [ ] Add payment data to cells
        - [ ] Add source badge with correct class
        - [ ] Add status badge
        - [ ] Add receipt number input field
        - [ ] Add action button (Done or Update based on status)
        - [ ] Add payment number if grouped (1/2, 2/2)
        - [ ] Add grouped-payment class if multiple
    - [ ] Render pagination controls
    - [ ] Attach button listeners

Pagination Functions

- [ ] **FE-JS-7** Create `renderPaginationControls(totalItems, currentPage, totalPages)` function

    - [ ] Create Previous button
    - [ ] Create page number buttons (show 5 at a time)
    - [ ] Create Next button
    - [ ] Disable buttons appropriately
    - [ ] Attach click listeners

- [ ] **FE-JS-8** Create `changePage(newPage)` function
    - [ ] Validate page number
    - [ ] Update currentPage in appState
    - [ ] Re-render table

Receipt Submission Function

- [ ] **FE-JS-9** Create `handleReceiptSubmission(paymentId, source, sourceTable, isUpdate)` function
    - [ ] Get receipt number from input
    - [ ] Validate receipt number (not empty)
    - [ ] If update: Show confirmation dialog with old vs new number
    - [ ] Prepare receipt data object
    - [ ] Call `submitCashReceipt(receiptData)`
    - [ ] Reload payments on success
    - [ ] Show success message
    - [ ] Error handling: Show error alert
    - [ ] Prevent submission if already loading

Statistics Function

- [ ] **FE-JS-10** Create `updateCashReceiptStats()` function
    - [ ] Count total payments
    - [ ] Count processed payments (status = 'processed')
    - [ ] Count pending payments (status = 'pending')
    - [ ] Calculate total amount
    - [ ] Update DOM elements with numbers
    - [ ] Format amounts with commas

Export Function

- [ ] **FE-JS-11** Create `exportCashReceiptsToExcel()` function
    - [ ] Get filteredPayments array
    - [ ] Create CSV data structure
    - [ ] Add headers
    - [ ] Add data rows
    - [ ] Create Blob
    - [ ] Create download link
    - [ ] Trigger download
    - [ ] Generate filename with current date

Helper Functions

- [ ] **FE-JS-12** Create `getSourceLabel(source)` function
    - [ ] Map 'supply' → 'Supply'

    - [ ] Map 'ugrai' → 'UGRAI'
    - [ ] Map 'ugrai_partial' → 'UGRAI (Partial)'

- [ ] **FE-JS-13** Create `getSourceBadgeClass(source)` function
    - [ ] Map source to appropriate CSS class

- [ ] **FE-JS-14** Create `showLoadingState(tableBody)` function
    - [ ] Display loading spinner in table

- [ ] **FE-JS-15** Create `showErrorInTable(tableBody, message)` function
    - [ ] Display error message in table

Event Listener Attachment

- [ ] **FE-JS-16** Create `attachReceiptButtonListeners()` function
    - [ ] Find all submit-receipt-btn elements
    - [ ] Attach click listeners wrapped in try-catch
    - [ ] Find all update-receipt-btn elements
    - [ ] Attach click listeners wrapped in try-catch
    - [ ] Prevent event bubbling

Tab System Integration

- [ ] **FE-JS-17** Integrate with existing tab switching logic
    - [ ] Add 'cash-receipt' case to tab switch handler
    - [ ] Show cash-receipt-dashboard-container
    - [ ] Hide other dashboard containers
    - [ ] Load payments when tab becomes active

Feature Flag Implementation

- [ ] **FE-JS-18** Add feature flag (optional but recommended)

```
const FEATURE_FLAGS = {
  cashReceiptEnabled: true // Set to false to disable feature
};
```

    - [ ] Check flag in initializeCashReceiptDashboard()

DOMContentLoaded Integration

- [ ] **FE-JS-19** Add initialization to DOMContentLoaded event

```
document.addEventListener('DOMContentLoaded', async () => {
  // ... existing code
  await initializeCashReceiptDashboard();
});
```

Phase 6: Integration & System Testing

Estimated Time: 5 hours 40 minutes

Role & Visibility Testing

- [ ] **TEST-1** Login as regular user (supply_in_progress role)
  - [ ] Verify Cash Receipt tab is hidden
  - [ ] Verify no console errors
- [ ] **TEST-2** Login as admin user
  - [ ] Verify Cash Receipt tab is visible
  - [ ] Click tab and verify dashboard loads
  - [ ] Check statistics display correctly
- [ ] **TEST-3** Login as accountant user
  - [ ] Verify Cash Receipt tab is visible
  - [ ] Verify full functionality works

Data Loading & Filtering Testing

- [ ] **TEST-4** Test initial data load
  - [ ] Verify payments appear from approved_bills
  - [ ] Verify payments appear from ugrai_payment_history
  - [ ] Check that same bill number appears multiple times if partial payment exists
- [ ] **TEST-5** Test date range filtering
  - [ ] Set start date: 2025-01-01, end date: 2025-01-31
  - [ ] Click "Apply Filter"
  - [ ] Verify only January payments shown
  - [ ] Click "Clear" and verify all payments shown again
- [ ] **TEST-6** Test status filtering
  - [ ] Click "Pending" filter
  - [ ] Verify only pending payments shown
  - [ ] Click "Processed" filter
  - [ ] Verify only processed payments shown

    - [ ] Click "All" filter
    - [ ] Verify all payments shown

Receipt Submission Testing

- [ ] **TEST-7** Test new receipt submission
    - [ ] Find a pending payment
    - [ ] Enter receipt number: "CASH-001"
    - [ ] Click "☑ Done"
    - [ ] Verify status changes to "Processed"
    - [ ] Verify button changes to "📝 Update"
    - [ ] Verify statistics update
- [ ] **TEST-8** Test receipt update
    - [ ] Find processed payment
    - [ ] Change receipt number to "CASH-002"
    - [ ] Click "📝 Update"
    - [ ] Verify confirmation dialog appears
    - [ ] Confirm and verify update succeeds
- [ ] **TEST-9** Test validation
    - [ ] Try submitting empty receipt number
    - [ ] Verify error message appears
    - [ ] Verify submission prevented
- [ ] **TEST-10** Test duplicate receipt number
    - [ ] Assign "CASH-003" to payment A
    - [ ] Try assigning "CASH-003" to payment B
    - [ ] Verify error message about duplicate

Cross-User Collaboration Testing

- [ ] **TEST-11** Test shared data access
    - [ ] User A assigns receipt "CASH-100" to Bill #123
    - [ ] Logout and login as User B (accountant)
    - [ ] Verify User B sees Bill #123 with receipt "CASH-100"
    - [ ] User B edits to "CASH-101"
    - [ ] Verify update succeeds
    - [ ] Login back as User A
    - [ ] Verify User A sees "CASH-101"

Export Testing

- [ ] **TEST-12** Test Excel export
    - [ ] Apply filters (date + status)

    - ☐ Click "📊 Export to Excel"
    - ☐ Verify file downloads
    - ☐ Open file and verify data accuracy
    - ☐ Check column headers are correct

Pagination Testing

- ☐ **TEST-13** Test pagination (if 50+ payments exist)
    - ☐ Verify only 50 items on page 1
    - ☐ Click "Next" and verify page 2 loads
    - ☐ Click "Previous" and verify page 1 loads
    - ☐ Click specific page number

Visual Grouping Testing

- ☐ **TEST-14** Test payment grouping
    - ☐ Find bill with multiple payments (partial payment case)
    - ☐ Verify "1/2" and "2/2" indicators appear
    - ☐ Verify blue left border on grouped rows

Performance Testing

- ☐ **TEST-15** Test with large dataset (100+ payments)
    - ☐ Load Cash Receipt tab
    - ☐ Measure load time (should be < 2 seconds)
    - ☐ Apply filters
    - ☐ Verify filtering is instant (< 500ms)
    - ☐ Check for UI freezing or lag

Error Handling Testing

- ☐ **TEST-16** Test network error handling
    - ☐ Disable network (DevTools → Network → Offline)
    - ☐ Try loading payments
    - ☐ Verify error message displays
    - ☐ Verify no console errors crash the app
- ☐ **TEST-17** Test invalid data handling
    - ☐ Manually corrupt payment data in console
    - ☐ Verify graceful error handling
    - ☐ Verify user sees helpful error message

Browser Compatibility Testing

- ☐ **TEST-18** Test in Chrome
- ☐ **TEST-19** Test in Firefox

- [ ] **TEST-20** Test in Edge

Mobile Responsiveness Testing

- [ ] **TEST-21** Test on mobile viewport (DevTools)
  - [ ] Verify table is readable (scroll or stack)
  - [ ] Verify buttons are tappable
  - [ ] Verify filters work on mobile

Phase 7: Deployment

Estimated Time: 5 hours 35 minutes

Pre-Deployment

- [ ] **DEP-1** Create SQL deployment script file
  - [ ] Combine all SQL statements (table, indexes, policies, functions)
  - [ ] Add comments explaining each section
  - [ ] Test script on local/staging database first
- [ ] **DEP-2** Create backup of production database
  - [ ] Verify backup is complete
  - [ ] Test restore procedure on staging

Staging Deployment

- [ ] **DEP-3** Deploy database to staging environment
  - [ ] Run SQL deployment script
  - [ ] Verify table created
  - [ ] Verify indexes created
  - [ ] Verify RLS policies active
  - [ ] Verify function exists
- [ ] **DEP-4** Deploy code to staging
  - [ ] Push changes to staging branch
  - [ ] Verify deployment successful
  - [ ] Check for console errors
- [ ] **DEP-5** Test in staging environment (run TEST-1 through TEST-21)
  - [ ] Complete all integration tests
  - [ ] Fix any bugs found
  - [ ] Re-test after fixes

Production Deployment (Feature Disabled)

- ☐ **DEP-6** Set feature flag to FALSE (if using)

```
const FEATURE_FLAGS = {
  cashReceiptEnabled: false
};
```

- ☐ **DEP-7** Deploy database to production
  - ☐ Run SQL deployment script in production
  - ☐ Verify no errors
  - ☐ Verify existing functionality unaffected
- ☐ **DEP-8** Deploy code to production
  - ☐ Merge feature branch to main
  - ☐ Deploy to production
  - ☐ Verify deployment successful
  - ☐ Monitor error logs (should be none)

Limited Production Testing

- ☐ **DEP-9** Enable feature flag for testing

```
const FEATURE_FLAGS = {
  cashReceiptEnabled: true
};
```

- ☐ **DEP-10** Test with 2-3 admin accounts
  - ☐ Process 10-20 real cash receipts
  - ☐ Verify data accuracy in database
  - ☐ Verify export functionality
  - ☐ Monitor system performance
- ☐ **DEP-11** Run post-deployment verification queries

```
-- Check cash_receipts table population
SELECT COUNT(*) FROM cash_receipts;

-- Check processed vs pending
SELECT status, COUNT(*) FROM cash_receipts GROUP BY status;

-- Check by source
SELECT payment_source, COUNT(*) FROM cash_receipts GROUP BY
payment_source;
```

```
-- Recent submissions
SELECT * FROM cash_receipts ORDER BY created_at DESC LIMIT 20;
```

Full Production Release

- ☐ **DEP-12** Confirm all tests passed
- ☐ **DEP-13** Keep feature enabled permanently
- ☐ **DEP-14** Send announcement to admin/accountant users
- ☐ **DEP-15** Provide quick reference guide
- ☐ **DEP-16** Schedule 30-minute training session (optional)
- ☐ **DEP-17** Monitor usage for first week
- ☐ **DEP-18** Gather user feedback
- ☐ **DEP-19** Address any issues or feature requests

Rollback Procedures (If Needed)

Level 1: Instant Disable

- ☐ **ROLLBACK-1** Set feature flag to false
- ☐ **ROLLBACK-2** Reload page to hide feature
- ☐ **ROLLBACK-3** Communicate to users

Level 2: Code Rollback

- ☐ **ROLLBACK-4** Git revert to previous commit
- ☐ **ROLLBACK-5** Redeploy previous version
- ☐ **ROLLBACK-6** Verify rollback successful

Level 3: Database Cleanup (Only if absolutely necessary)

- ☐ **ROLLBACK-7** Drop function: `DROP FUNCTION get_all_cash_payments;`
- ☐ **ROLLBACK-8** Drop table: `DROP TABLE cash_receipts CASCADE;`

Documentation Tasks

- ☐ **DOC-1** Create user guide for admin/accountant users
  - ☐ How to access the feature
  - ☐ How to filter payments
  - ☐ How to assign receipt numbers
  - ☐ How to export data
  - ☐ Troubleshooting common issues
- ☐ **DOC-2** Create screenshots or GIFs for guide
  - ☐ Dashboard overview

    - ☐ Date filtering
    - ☐ Receipt submission
    - ☐ Export process

- ☐ **DOC-3** Update main system documentation
    - ☐ Add Cash Receipt feature to feature list
    - ☐ Update architecture diagrams

- ☐ **DOC-4** Create FAQ document
    - ☐ Common questions and answers
    - ☐ Known limitations
    - ☐ Future enhancements

☑ Final Verification Checklist

Functional Verification

- ☐ Tab visible only to admin and accountant users
- ☐ All cash payments from approved_bills display
- ☐ All cash payments from ugrai_payment_history display
- ☐ Date range filtering works accurately
- ☐ Status filtering (all/pending/processed) works
- ☐ Receipt numbers can be submitted
- ☐ Receipt numbers can be updated
- ☐ Visual indicators show correct status
- ☐ Statistics update in real-time
- ☐ Export to Excel generates accurate files
- ☐ Pagination works correctly
- ☐ Visual grouping works for multiple payments per bill
- ☐ Cross-user collaboration works (shared data)

Technical Verification

- ☐ Database table created without errors
- ☐ RLS policies protect data correctly
- ☐ SQL function returns data within 2 seconds
- ☐ No console errors in browser
- ☐ No impact on existing functionality
- ☐ Code follows existing patterns and style
- ☐ Feature flag allows instant enable/disable (if implemented)

Performance Verification

- ☐ Initial load < 2 seconds (for 500 payments)
- ☐ Filter application < 500ms
- ☐ Receipt submission < 1 second

- ☐ Export to Excel < 3 seconds (for 1000 payments)
- ☐ No memory leaks

Security Verification

- ☐ RLS policies prevent unauthorized access
- ☐ Role-based access works correctly
- ☐ Input validation prevents injection attacks
- ☐ XSS prevention (escapeHtml) working
- ☐ Audit trail tracks all changes (user_id field)

📊 Progress Tracker

Start Date: _____
Target Completion Date: _____
Actual Completion Date: _____

Daily Progress Log:

Day 1:

- Tasks Completed: _____ / 71
- Hours Spent: _____
- Blockers: _____
- Notes: _____

Day 2:

- Tasks Completed: _____ / 71
- Hours Spent: _____
- Blockers: _____
- Notes: _____

Day 3:

- Tasks Completed: _____ / 71
- Hours Spent: _____
- Blockers: _____
- Notes: _____

Day 4:

- Tasks Completed: _____ / 71
- Hours Spent: _____
- Blockers: _____
- Notes: _____

🎯 Success Criteria

Feature is considered complete when:

- [ ] All 71 tasks checked off
- [ ] All tests passing
- [ ] No console errors
- [ ] User feedback positive
- [ ] Performance metrics met
- [ ] Documentation complete
- [ ] Deployed to production successfully

Document Status: Ready for Use
Last Updated: February 20, 2026
Maintained By: Development Team